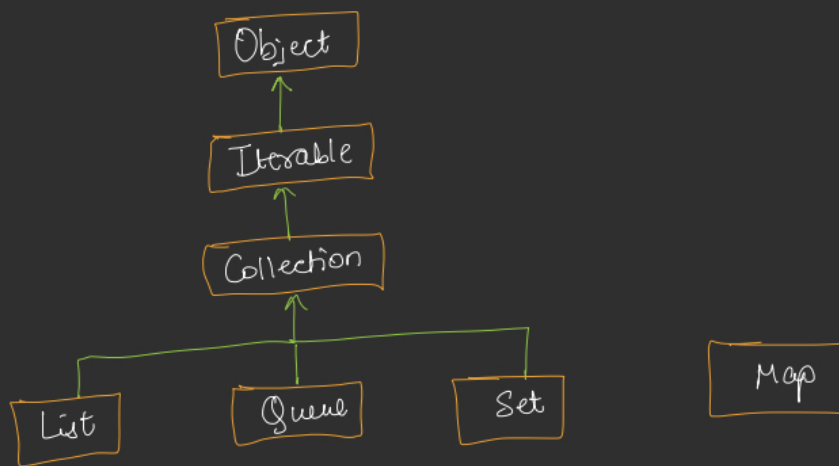




Collection

- ↳ ArrayList
- ↳ LinkedList
- ↳ Stack / Queue
- ↳ Set / Map

`list<Integer> list = new ArrayList<>();`

i

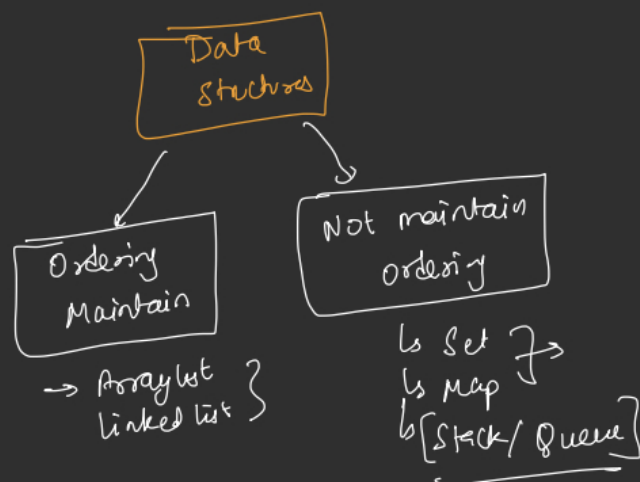
2	3	5	9	7	1
---	---	---	---	---	---

 = 6

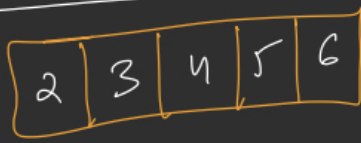
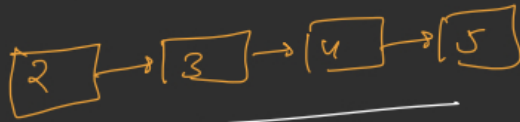
_{0 1 2 3 4 5}
_i

for(i=0; i < list.size(); i++) {
 System.out.println(list.get(i));
 }

[2, 3, 5, 9, 7, 1]

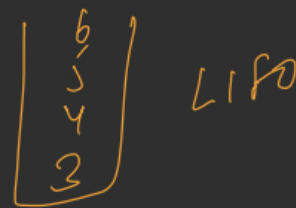
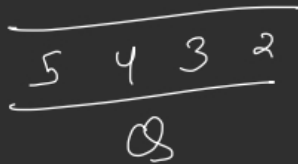


`list<Integer> list = new LinkedList();`



```
for (i=0; i < list.size(); i++) {  
    list.get(i);  
}
```

Collection $\rightarrow$ Iterable
 $\hookrightarrow$ index based (x)



Stack

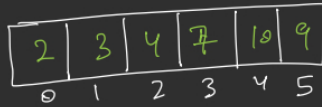


2

Iterable

$\hookrightarrow$ It is the root interface that represent any object whose element can be traversed one by one.

Arraylist



LinkedList

Set

Queue



hasNext()
next()

Iterable

↳ iterator()

Iterator

↳ hasNext()
↳ next()

interface Iterable of

Iterator iterator();

}

ArrayList

Iterator iterator() {

return new
ArrayListIterator();

}

interface Iterator of

hasNext()
next();

class ArrayListIterator

implements Iterator of

hasNext() { }

next() { }

Iterator

ArrayListIterator

list list = new ArrayList<>();

Iterator it =

list.iterator();

it.hasNext()

it.next()

How Different Collection Conceptually implement Iterable / Iterator

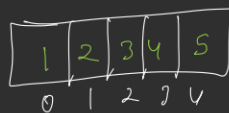
ArrayList



```

class ArrayList implements Iterable {
    private Integer[] arr;
    private int size;

    @Override
    public Iterator iterator() {
        return new ArrayListIterator();
    }
}
    
```



size = 5

```

class ArrayListIterator
    implements Iterator {
    int pos = 0;

    @Override
    public boolean hasNext() {
        // ...
    }

    @Override
    public Integer next() {
        // ...
    }
}
    
```

```

class ArrayList implements Iterable {
    private Integer[] arr;
    private int size;

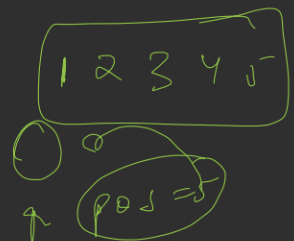
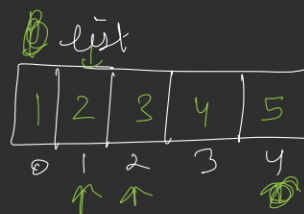
    @Override
    public Iterator iterator() {
        return new ArrayListIterator();
    }
}
    
```

```

class ArrayListIterator
    implements Iterator {
    int pos = 0;

    @Override
    public boolean hasNext() {
        return (pos < size);
    }

    @Override
    public Integer next() {
        return arr[pos++];
    }
}
    
```



```

main() {
    Iterator it = list.iterator();
}
    
```

```

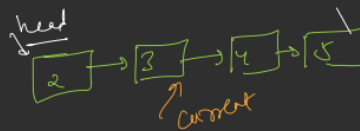
while (it.hasNext()) {
    // ...
    (it.next());
}
    
```

Class LinkedList implements Iterable {

static class Node {
int data;
Node next;

Node head;

@Override
Iterator iterator() {
return new LinkedListIterator();
}



class LinkedListIterator

implements Iterator {

Node current = head;

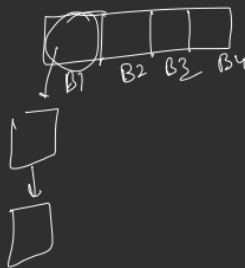
@Override
boolean hasNext() {
return current != null;
}

@Override
Integer next() {
int data = current.data;
current = current.next;
return data;
}

while (it.hasNext())
it.next();

Set / Map

↳ Hashes



TreeSet / TreeMap

↳ BST

↳ inorder()

Queue / Stack

↳ ArrayList

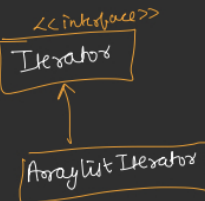
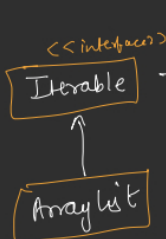
Iterable

↳ iterator()
↳ forEach()
↳ splitIterator()

Iterator

↳ hasNext()
↳ next()
↳ remove()

↳ forEachRemaining()
functional interface



① Separation of Concern

②

Class AL implements Iterator {
hasNext()
next()
}

```

class MyArrayList {
    int[] arr = {10, 20, 30};
    int size = 3;
    private int pos = 0;
    public boolean hasNext() {
        return pos < size;
    }
    public boolean next() {
        return arr[pos++];
    }
}

```

→ void reset() { pos = 0; }

10 | 20 | 30
 pos pos
 d = new MyArrayList();
 d.reset();

a = 10
 b = 20

```

while (d.hasNext()) {
    int a = d.next();
    while (d.hasNext()) {
        int b = d.next();
        (a, b);
    }
}

```

10	20
10	30